```
    3.times do
      number = rand(10) + 1
      puts "#{number}..."
    end
    "#{person} got a #{number}"
  end
end
```

You can also develop this code in bite-sized steps. Start by writing regular methods; then, when you're confident that your code is working, refactor the methods to a method_missing. This way, you won't inadvertently hide a bug behind a Ghost Method.

Blank Slates

Where you and Bill learn to avoid another common method_missing trap.

Once you get back from lunch, you find an unexpected problem waiting for you at the office. The developer who wrote the reporting application stumbled upon what he thinks is "the strangest bug ever": the Computer class can't retrieve information about the workstations' displays. All the other methods work fine, but Computer#display doesn't.

You try the display method in irb, and sure enough it fails:

```
my_computer = Computer.new(42, DS.new)
my_computer.display     # => nil
```

Why does Computer#display return nil? You triple-check the code and the back-end data source, but everything seems to be fine. Bill has a sudden insight, and he lists the instance methods of Object that begin with a *d*:

```
Object.instance_methods.grep /^d/   # => [:dup, :display, :define_singleton_method]
```

It seems that Object defines a method named display (a seldom-used method that prints an object on a port and always returns nil). Computer inherits from Object, so it gets the display method. The call to Computer#display finds a real method by that name, so it never lands on method_missing. You're calling a real, live method instead of a *Ghost Method* (57).

This problem crops up with *Dynamic Proxies* (60). When the name of a Ghost Method clashes with the name of a real, inherited method, the latter wins.

If you don't need the inherited method, you can fix the problem by removing it. While you're at it, you might want to remove most methods from the class, preventing such name clashes from ever happening again. A skinny class with a minimal number of methods is called a *Blank Slate*. As it turns out, Ruby has a ready-made Blank Slate for you to use.

Spell: Blank Slate